

CPSC 340: Machine Learning and Data Mining

Gradient Descent

Last Time: Change of Basis

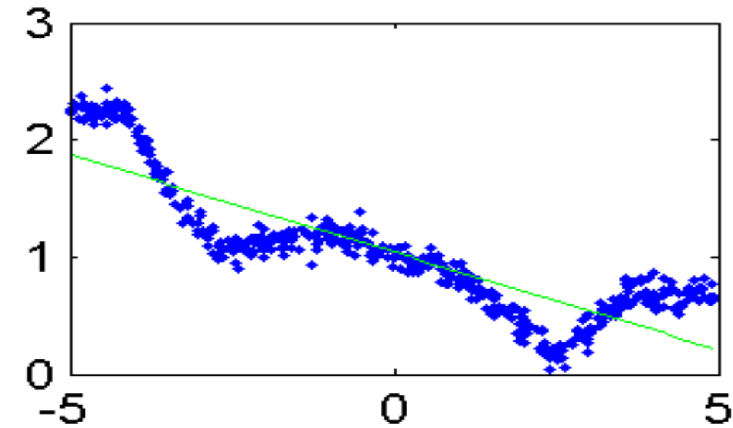
- Last time we discussed **change of basis**:
 - E.g., **polynomial basis**:

Replace $X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ with $Z = \begin{bmatrix} 1 & x_1 & (x_1)^2 & \dots & (x_1)^p \\ 1 & x_2 & (x_2)^2 & \dots & (x_2)^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & (x_n)^2 & \dots & (x_n)^p \end{bmatrix}$

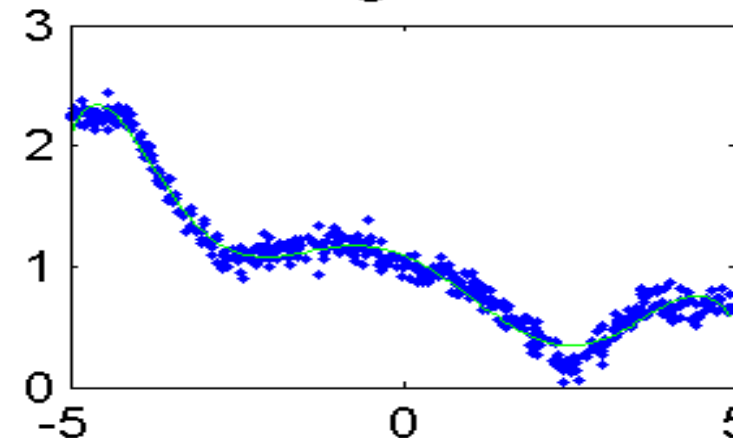
- You can **fit non-linear models** with linear regression.

$$\hat{y}_i = v^T z_i = w_0 + w_1 x_i + w_2 x_i^2 + w_3 x_i^3 + \dots + w_p x_i^p$$

- Just treat 'Z' as your data, then fit linear model.



Degree 7



Optimization Terminology

- When we **minimize** or **maximize** a function we call it “**optimization**”.
 - In least squares, we want to solve the “**optimization problem**”:

$$\min_{w \in \mathbb{R}^d} \frac{1}{2} \sum_{i=1}^n (w^T x_i - y_i)^2$$

- The function being optimized is called the “**objective**”.
 - Also sometimes called “loss” or “cost”, but these have different meanings in ML.
- The set over which we search for an optimum is called the **domain**.
- Often, instead of the minimum objective value, you want a **minimizer**.
 - A set of **parameters** ‘w’ that achieves the minimum value.

Discrete vs. Continuous Optimization

- We have seen examples of **continuous optimization**:



C
[Signature]

- Least squares:

- Domain is the real-valued set of parameters 'w'.
- Objective is the sum of the squared training errors.

- We have seen examples of **discrete optimization**:

- Fitting decision stumps:

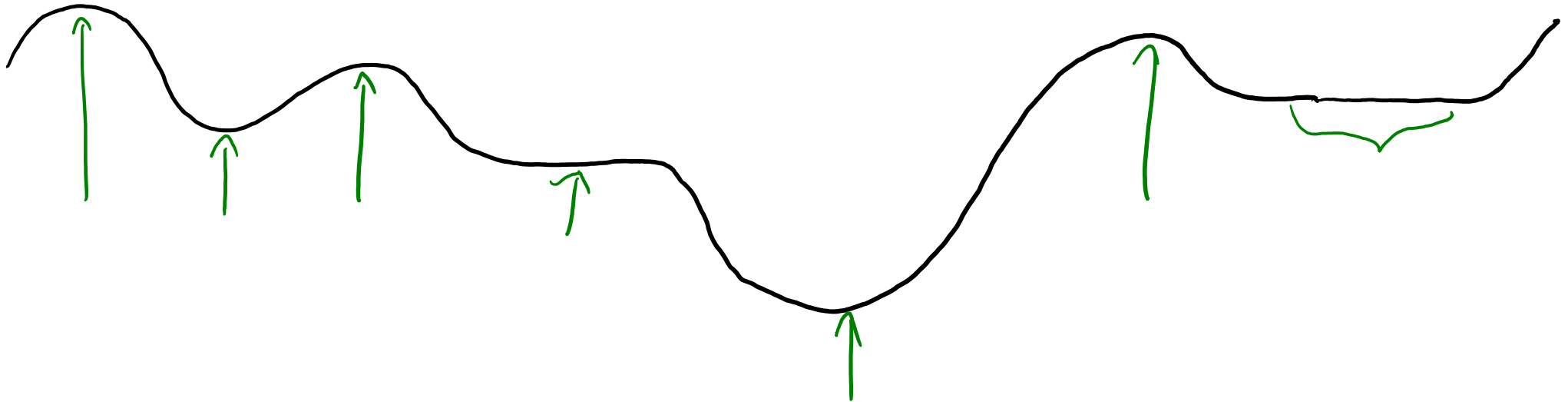
- Domain is the finite set of unique rules.
- Objective is the number of classification errors (or infogain).

- We have also seen a **mixture** of discrete and continuous:

- K-means: clusters are discrete and means are continuous.

Stationary/Critical Points

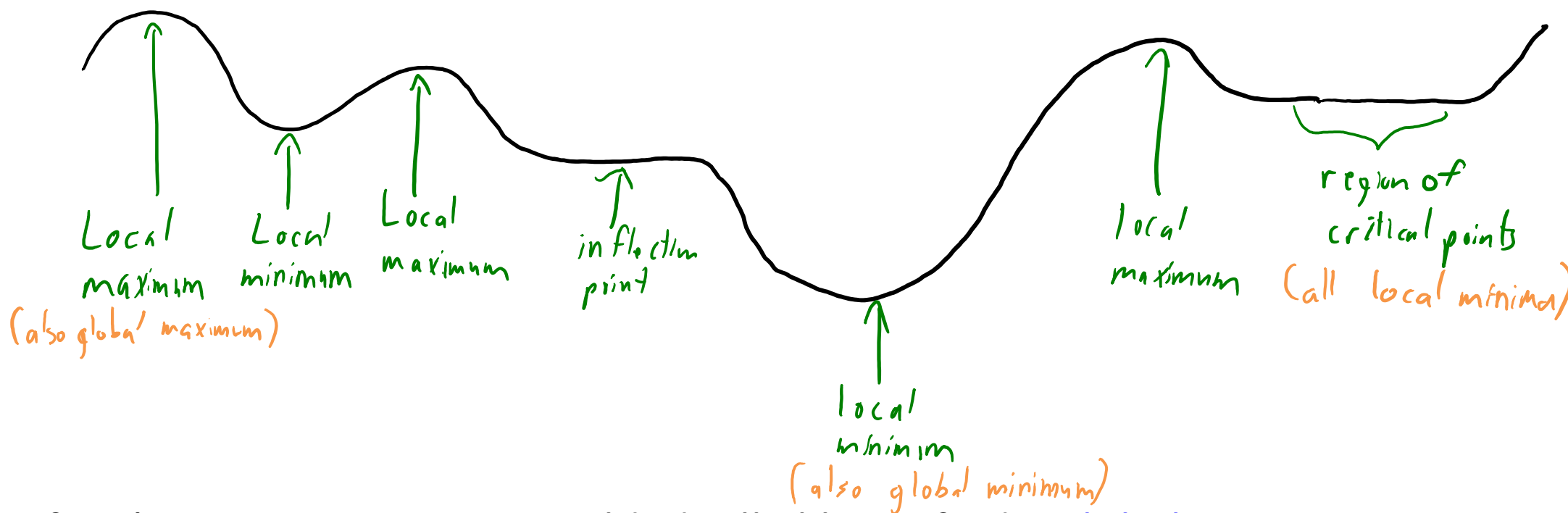
- A 'w' with $\nabla f(w) = 0$ is called a **stationary point** or **critical point**.
 - The slope is zero so the tangent plane is “flat”.



Critical points

Stationary/Critical Points

- A 'w' with $\nabla f(w) = 0$ is called a **stationary point** or **critical point**.
 - The slope is zero so the tangent plane is “flat”.



- If we're minimizing, we would ideally like to find a **global minimum**.
 - But **for some problems the best we can do is find a stationary point** where $\nabla f(w)=0$.

Motivation: Large-Scale Least Squares

- Normal equations find 'w' with $\nabla f(w) = 0$ in $O(nd^2 + d^3)$ time.

$$\underbrace{(X^T X)}_{O(nd^2)} \underbrace{w}_{O(nd)} = \underbrace{X^T y}_{O(nd)}$$

(matrix multiply) (matrix-vector)

$\nabla f(w)$

→ Solving a $d \times d$ system is $O(d^3)$

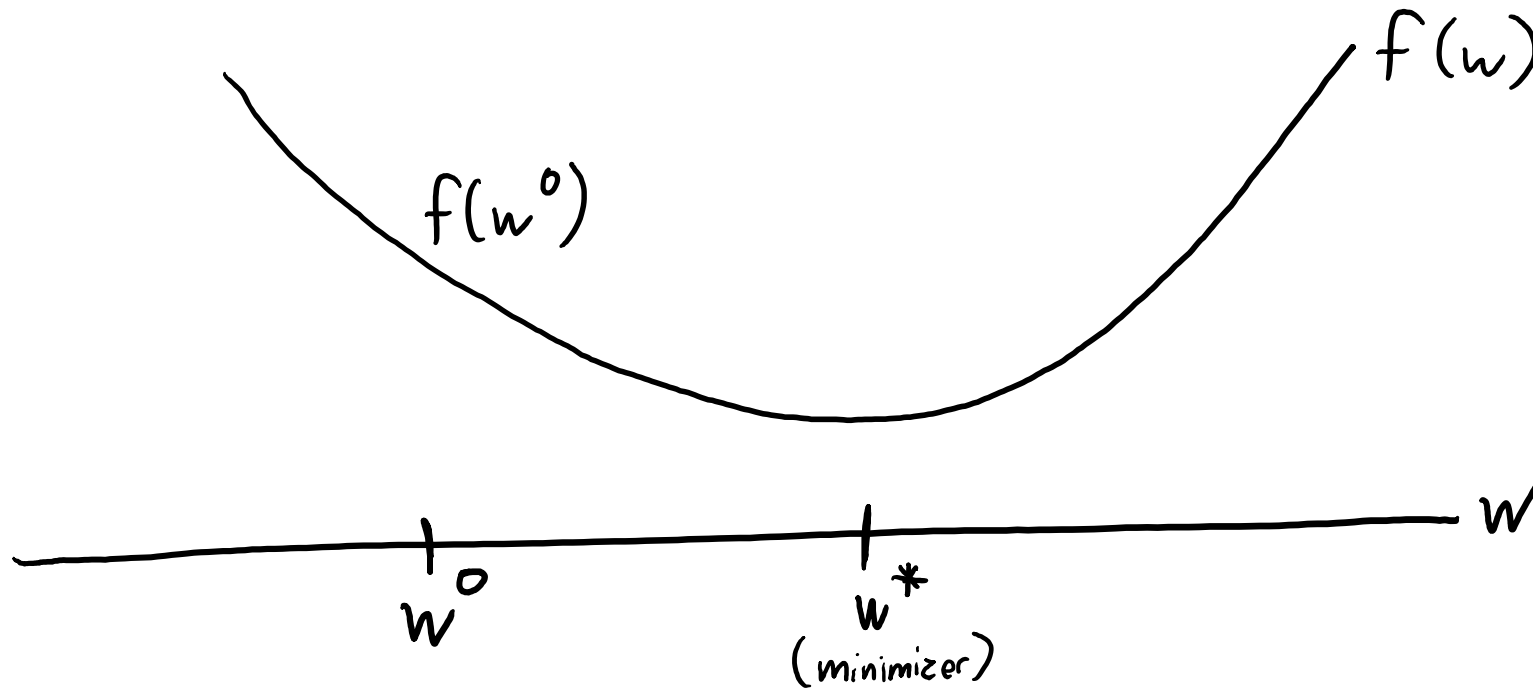
- Very slow if 'd' is large.
- Alternative when 'd' is large is gradient descent methods.
 - Probably the most important class of algorithms in machine learning.

Gradient Descent for Finding a Local Minimum

- Gradient descent is an iterative optimization algorithm:
 - It starts with a “guess” w^0 .
 - It uses the gradient $\nabla f(w^0)$ to generate a better guess w^1 .
 - It uses the gradient $\nabla f(w^1)$ to generate a better guess w^2 .
 - It uses the gradient $\nabla f(w^2)$ to generate a better guess w^3 .
 - ...
 - The limit of w^t as ‘t’ goes to ∞ has $\nabla f(w^t) = 0$.
- It converges to a global optimum if ‘f’ is “convex”.

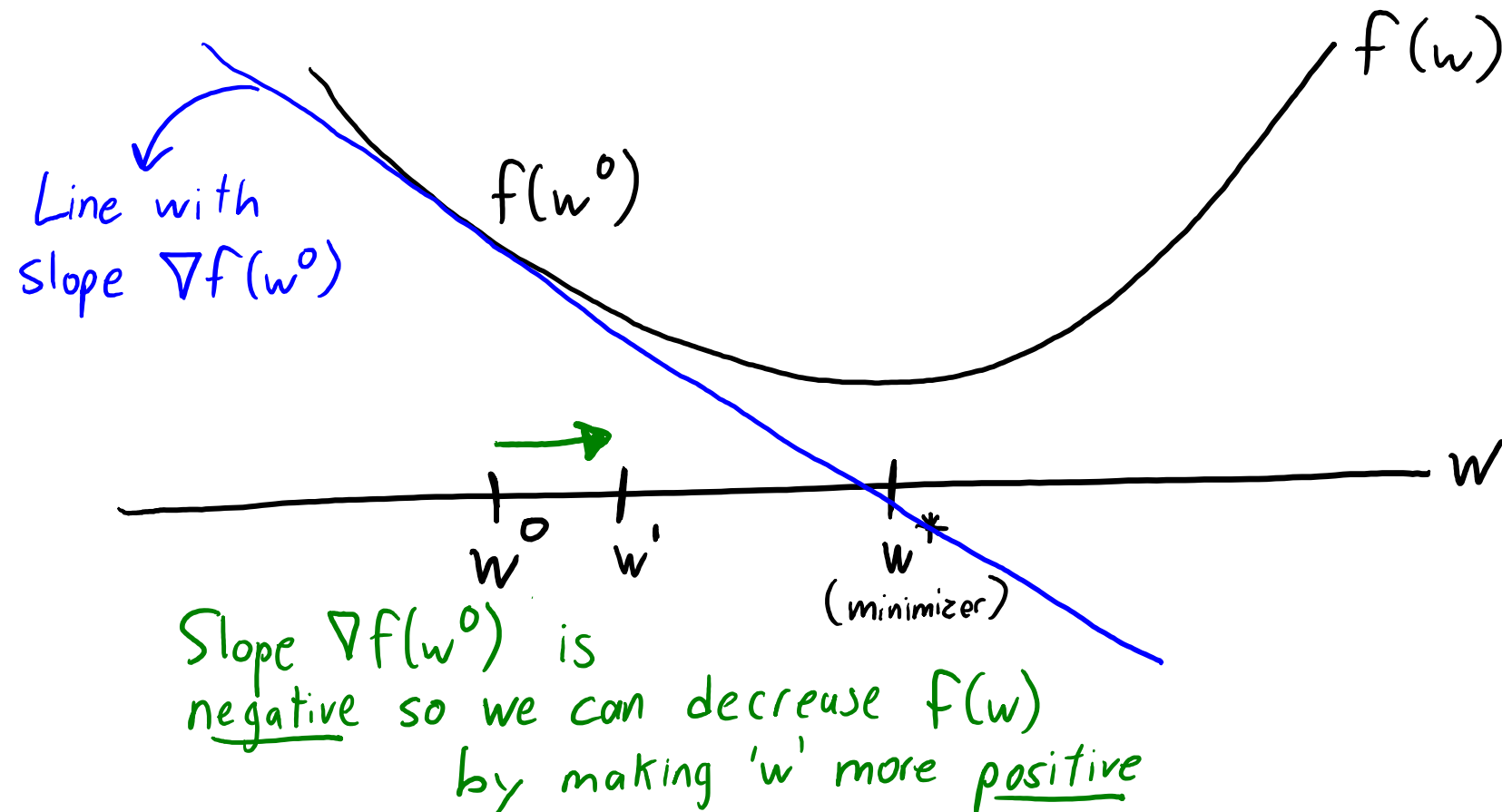
Gradient Descent for Finding a Local Minimum

- **Gradient descent** is based on a simple observation:
 - Give parameters ' w ', the **direction of largest decrease** is $-\nabla f(w)$.



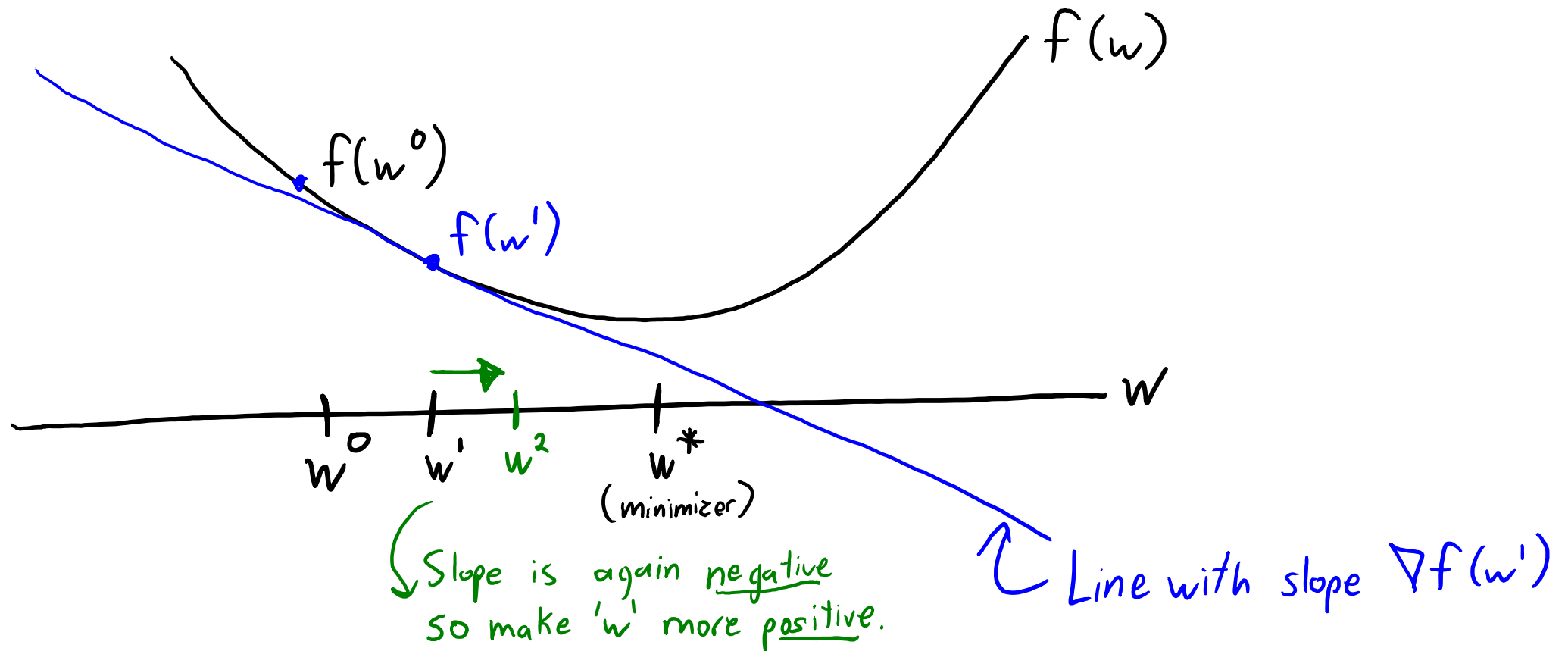
Gradient Descent for Finding a Local Minimum

- **Gradient descent** is based on a simple observation:
 - Give parameters ' w ', the **direction of largest decrease** is $-\nabla f(w)$.



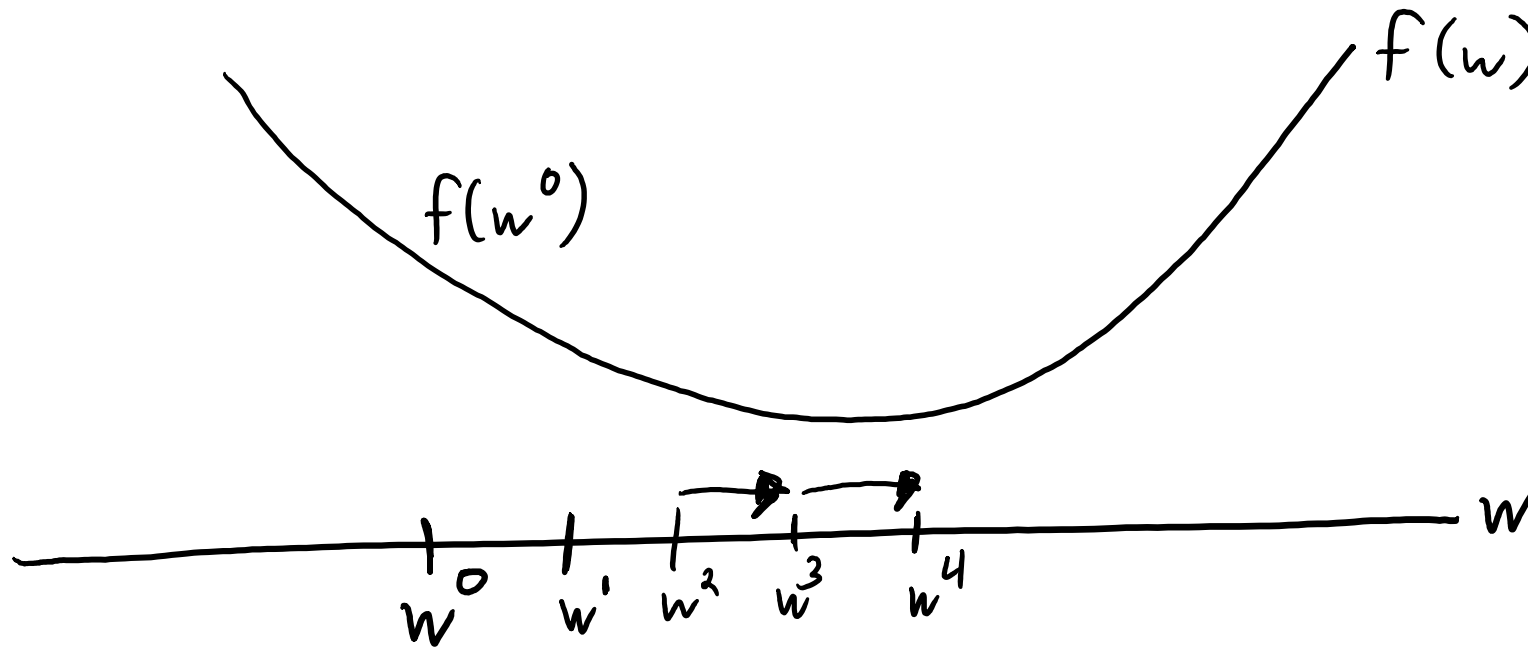
Gradient Descent for Finding a Local Minimum

- **Gradient descent** is based on a simple observation:
 - Give parameters ' w ', the **direction of largest decrease** is $-\nabla f(w)$.



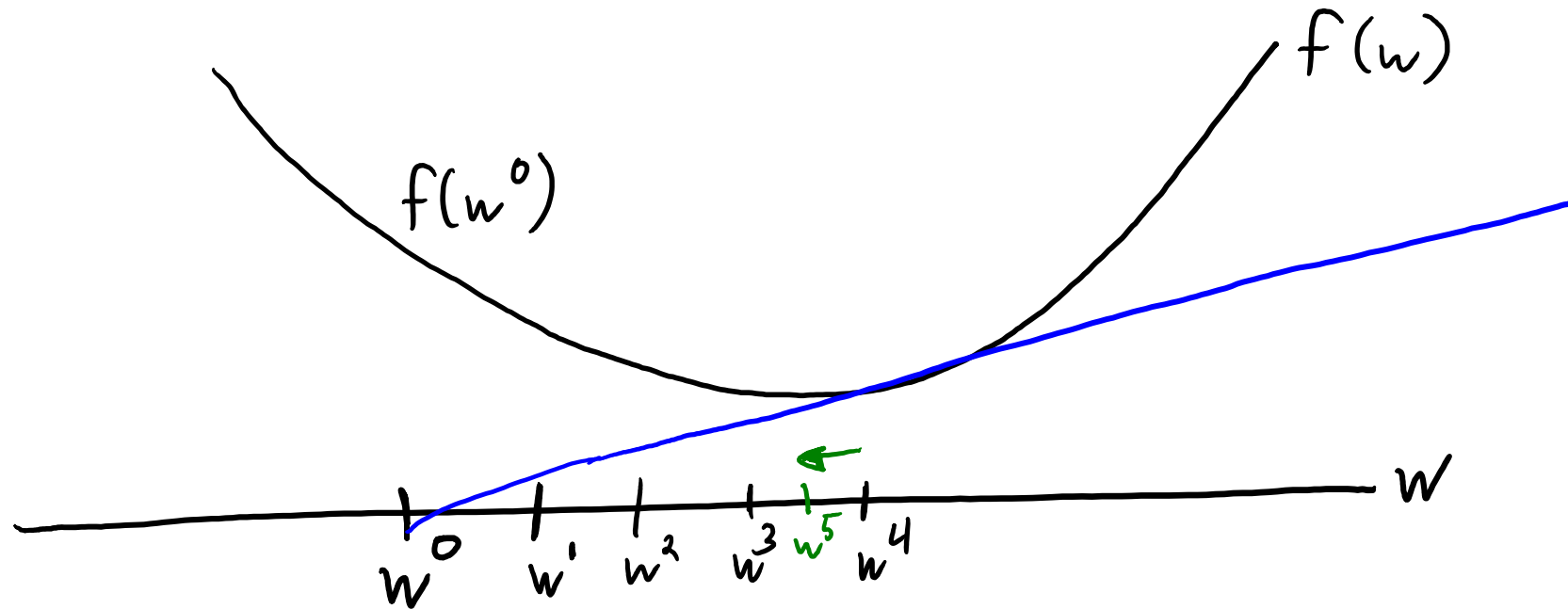
Gradient Descent for Finding a Local Minimum

- **Gradient descent** is based on a simple observation:
 - Give parameters ' w ', the **direction of largest decrease** is $-\nabla f(w)$.



Gradient Descent for Finding a Local Minimum

- **Gradient descent** is based on a simple observation:
 - Give parameters ' w ', the **direction of largest decrease** is $-\nabla f(w)$.



Now the slope $\nabla f(w^4)$ is positive
so we move in the negative direction.

Gradient Descent for Finding a Local Minimum

- We start with some initial guess, w^0 .
- Generate new guess by moving in the negative gradient direction:

$$w^1 = w^0 - \alpha^0 \nabla f(w^0)$$

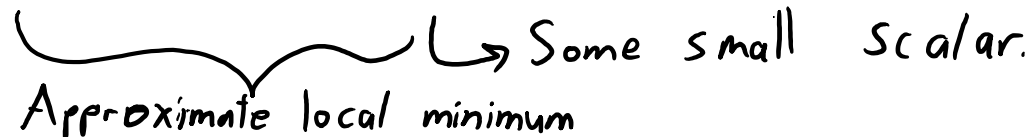
- The value α^0 is the “step size” (or “learning rate”).
- This decreases ‘f’ if the step size is small enough.
- Usually, we decrease α^0 if it increases ‘f’ (see “findMin”).
- Repeat to successively refine the guess:

$$w^{t+1} = w^t - \alpha^t \nabla f(w^t) \quad \text{for } t = 1, 2, 3, \dots$$

- Stop if not making progress or $\|\nabla f(w^t)\| \leq \epsilon$
- $\underbrace{\qquad\qquad\qquad}_{\text{Approximate local minimum}} \rightarrow \text{Some small scalar.}$

Gradient Descent for Finding a Local Minimum

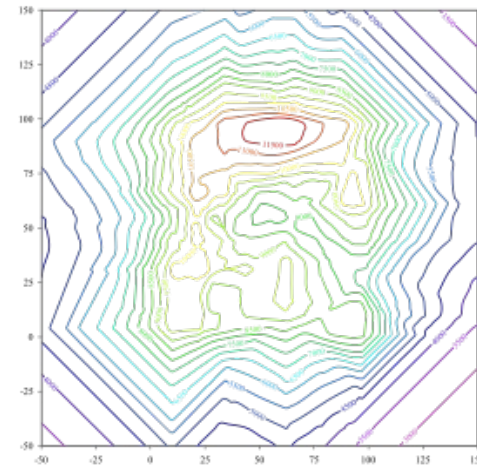
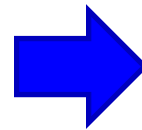
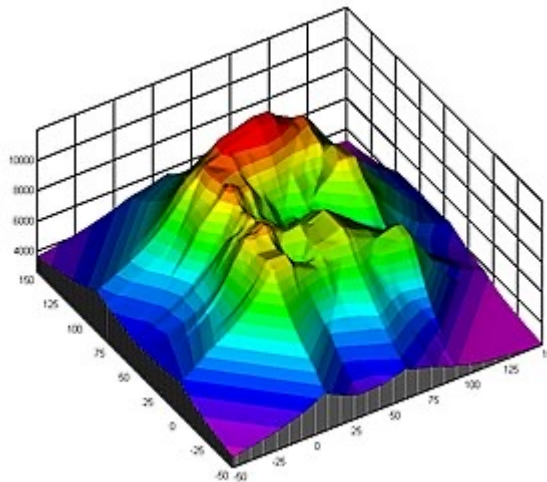
- Is gradient descent **guaranteed to work**?
 - Under weak conditions, **limit of $\nabla f(w^t)$ is 0** as iteration number “t” goes to ∞ .
 - Function ‘f’ is **continuously differentiable** (so gradients always exist).
 - Function ‘f’ is **bounded below** (so you cannot decrease ‘f’ to $-\infty$).
 - **Second derivatives of ‘f’ are bounded** (gradient cannot change too fast).
 - Guarantees some progress on each step (this assumption can be weakened).
 - We eventually use **small enough constant α^t** (many other strategies work too).
- How do we **decide when to stop**?
 - Check if: $\|\nabla f(w^t)\| \leq \epsilon$



Approximate local minimum Some small scalar.
 - Might also check $\|w^t - w^{t-1}\|$ and/or $|f(w^t) - f(w^{t-1})|$.
 - Checks of “progress”: if too small gradient descent may be **going slow**.

Digression: Contour Maps

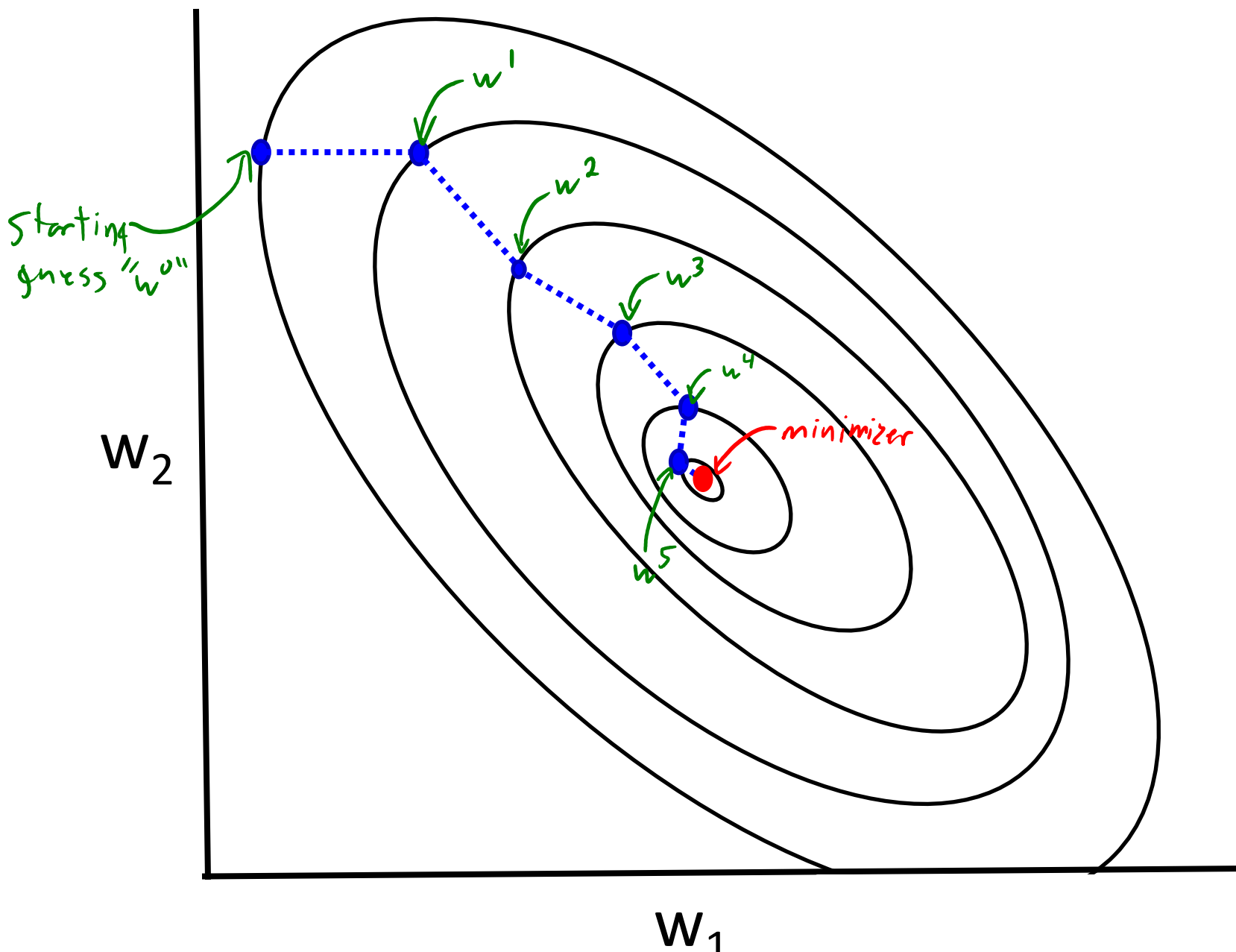
- To visualize gradient descent in 2D, we often use **contour lines**:
 - Figure where **each curve has same function value**.



- Often used to show elevation:

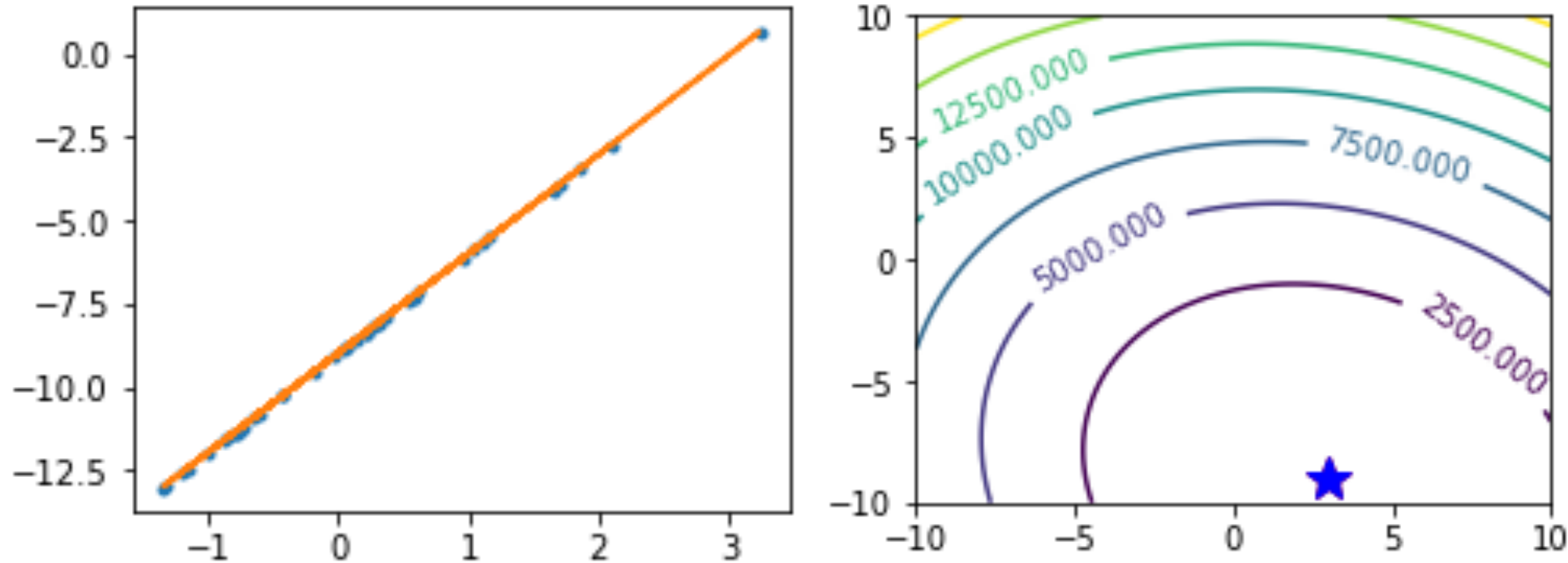


Gradient Descent in 2D



Data Space vs. Parameter Space

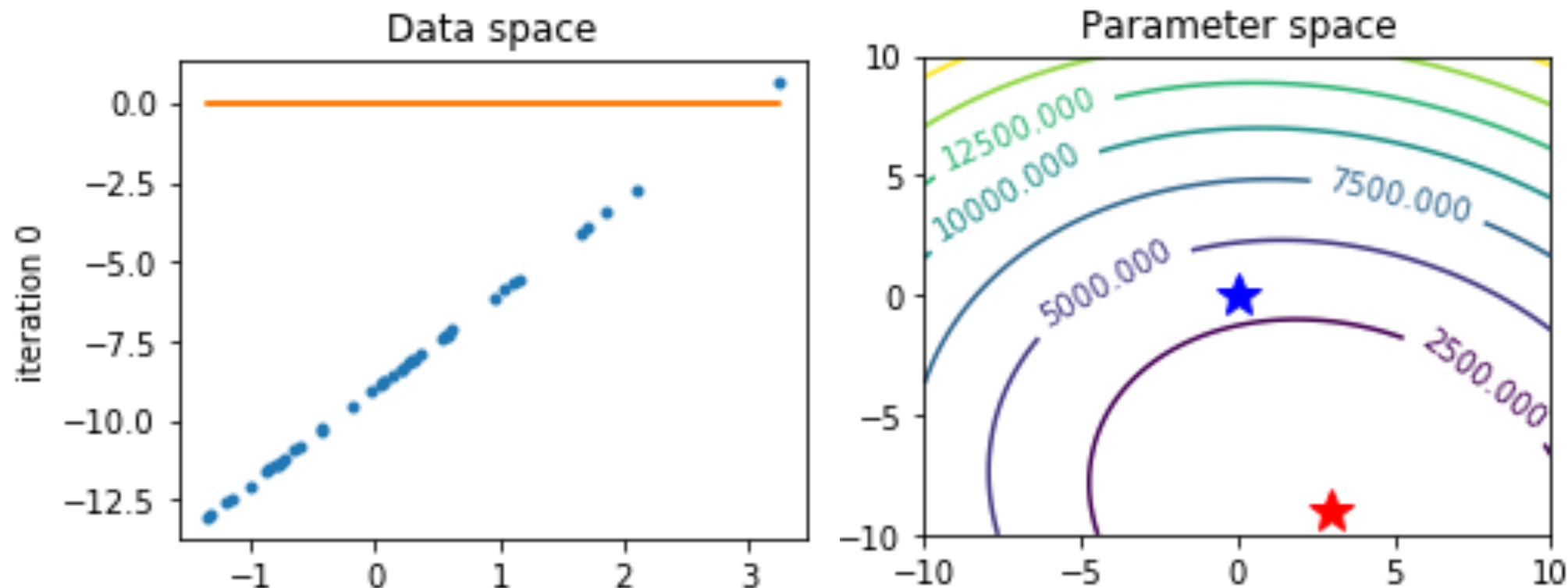
- Usual regression plot is in the “x vs. y” **data space** (left):



- On the right is plot of the “intercept vs. slope” **parameter space**.
 - Points in parameter space correspond to models (* is **least squares parameters**).

Gradient Descent in Data Space vs. Parameter Space

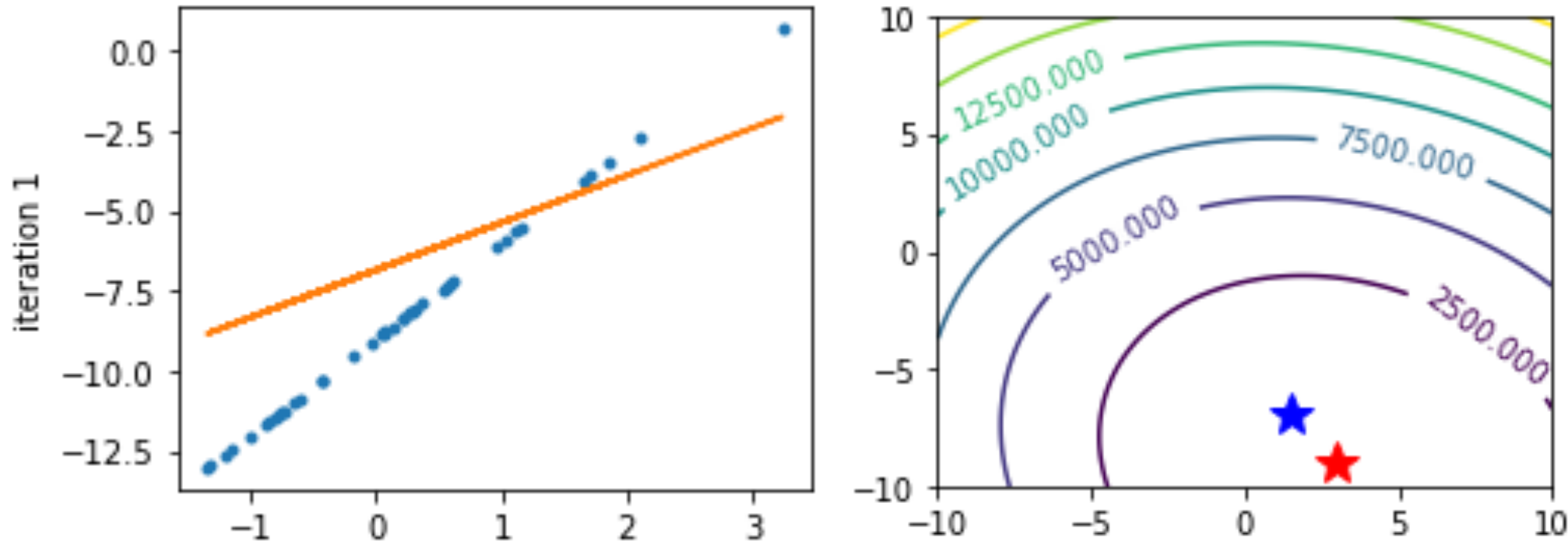
- Gradient descent starts with an initial guess in parameter space:



- And each iteration tries to move **guess** closer to **solution**.

Gradient Descent in Data Space vs. Parameter Space

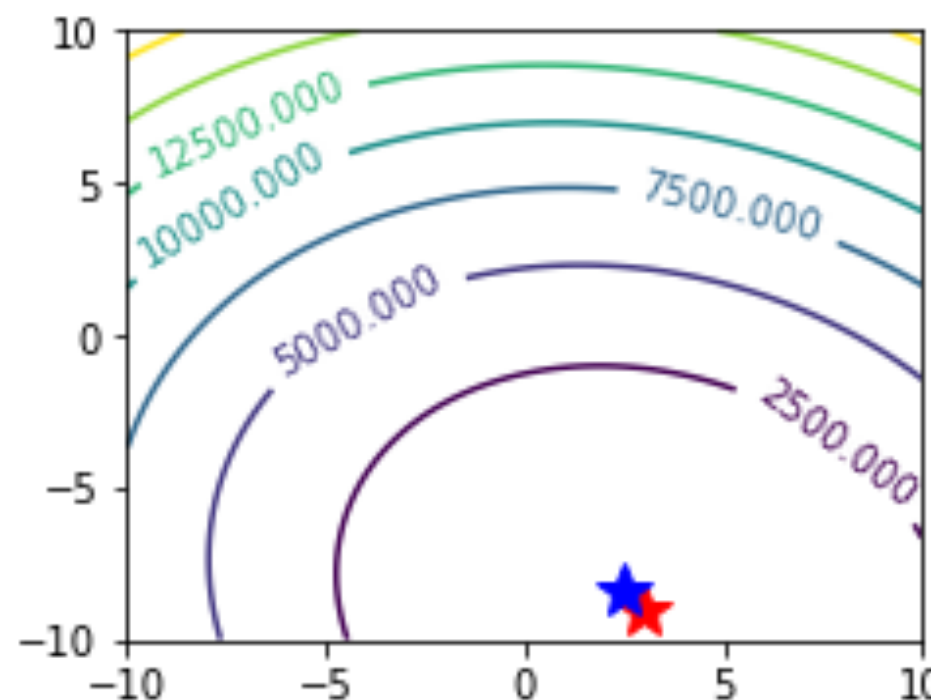
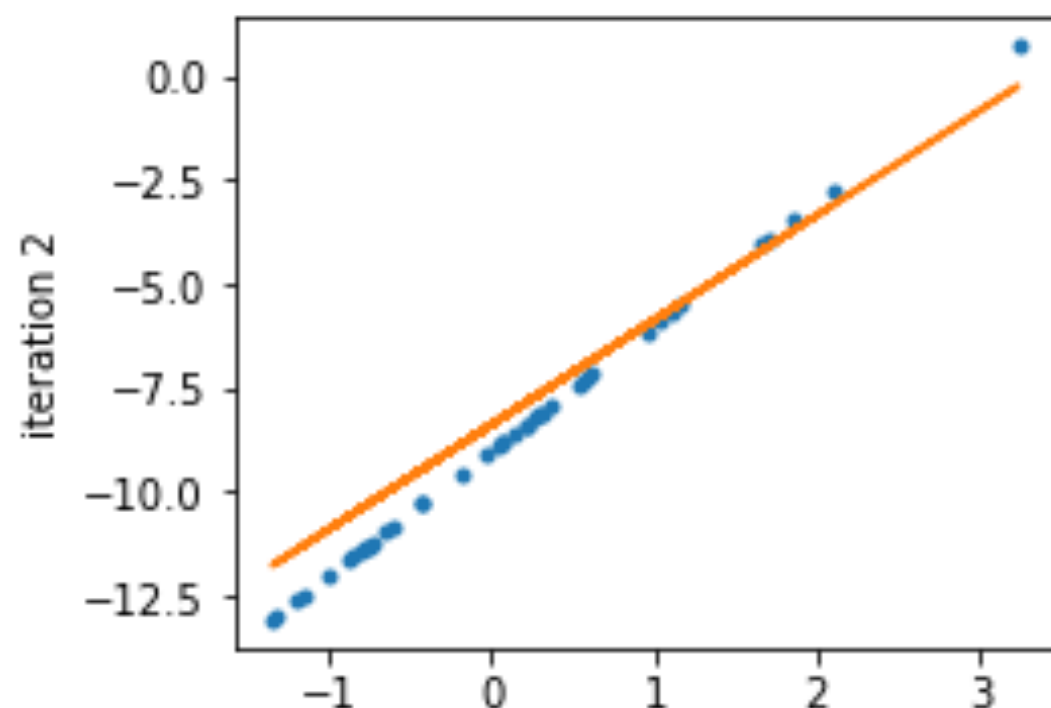
- Gradient descent starts with an initial guess in parameter space:



- And each iteration tries to move **guess** closer to **solution**.

Gradient Descent in Data Space vs. Parameter Space

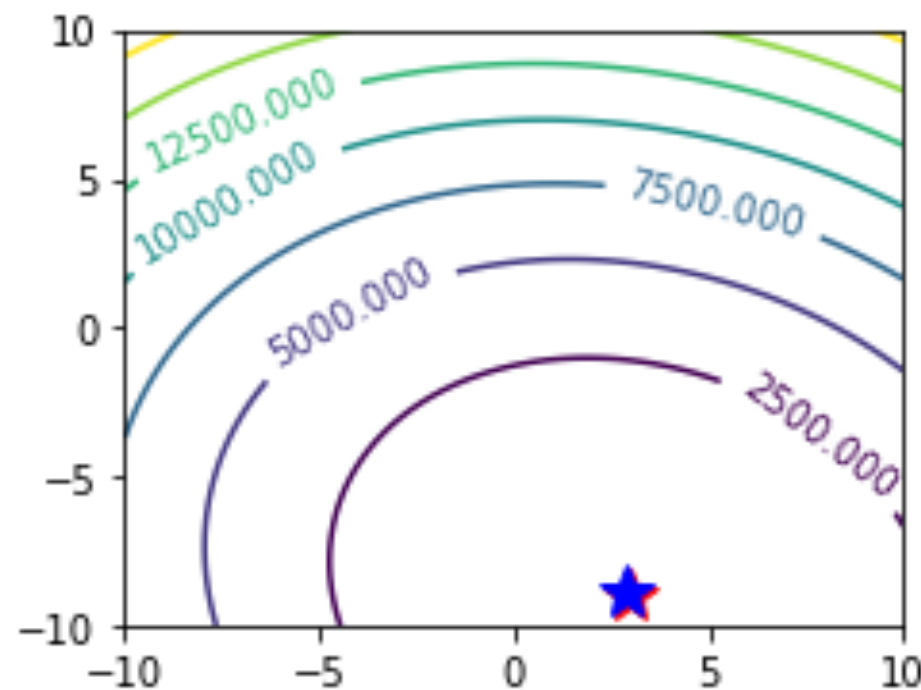
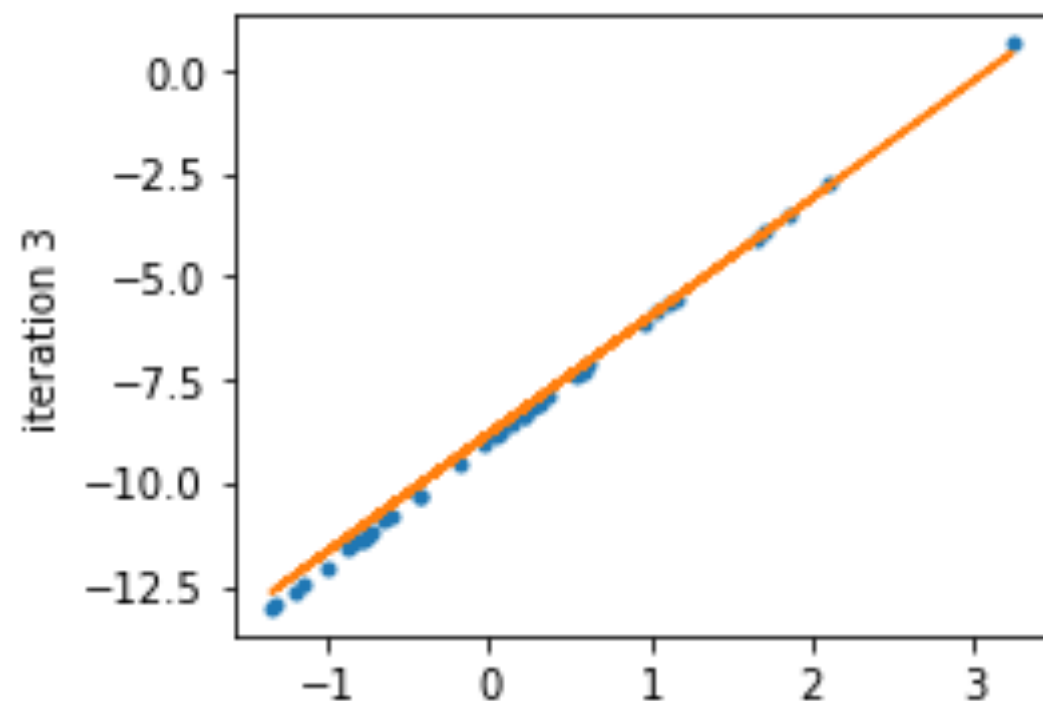
- Gradient descent starts with an initial guess in parameter space:



- And each iteration tries to move **guess** closer to **solution**.

Gradient Descent in Data Space vs. Parameter Space

- Gradient descent starts with an initial guess in parameter space:



- And each iteration tries to move **guess** closer to **solution**.

Gradient Descent for Least Squares

- The least squares objective and gradient:

$$f(w) = \frac{1}{2} \|Xw - y\|^2 \quad \nabla f(w) = X^T(Xw - y)$$

- Gradient descent iterations for least squares:

$$w^{t+1} = w^t - \alpha^t \underbrace{X^T(Xw^t - y)}_{\nabla f(w^t)}$$

- Cost of gradient descent iteration is $O(nd)$ (no need to form $X^T X$).

Bottleneck is computing $\nabla f(w^t) = X^T(Xw^t - y)$

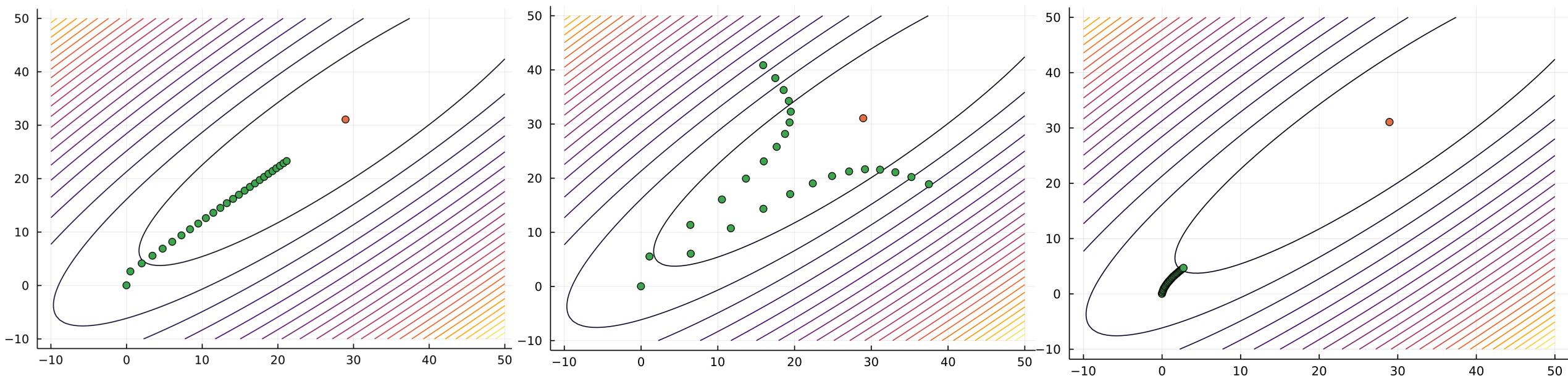
$\underbrace{\underbrace{X}_{O(nd)} \underbrace{(Xw^t - y)}_{O(n)}}_{O(nd)}$

Normal Equations vs. Gradient Descent

- Least squares via normal equations vs. gradient descent:
 - Normal equations cost $O(nd^2 + d^3)$.
 - Gradient descent costs $O(ndt)$ to run for 't' iterations.
 - Each of the 't' iterations costs $O(nd)$.
 - Gradient descent can be faster when 'd' is very large:
 - If solution is “good enough” for a 't' less than $\text{minimum}(d, d^2/n)$.
 - CPSC 5XX: 't' proportional to “condition number” of $X^T X$ (no direct 'd' dependence).
 - Normal equations only solve linear least squares problems.
 - Gradient descent solves many other problems.

Step Size Considerations

- The step size α^t must be set carefully for gradient descent to work.
 - If α^t is too large gradient descent may not converge.
 - If α^t is too small gradient descent may be too slow.



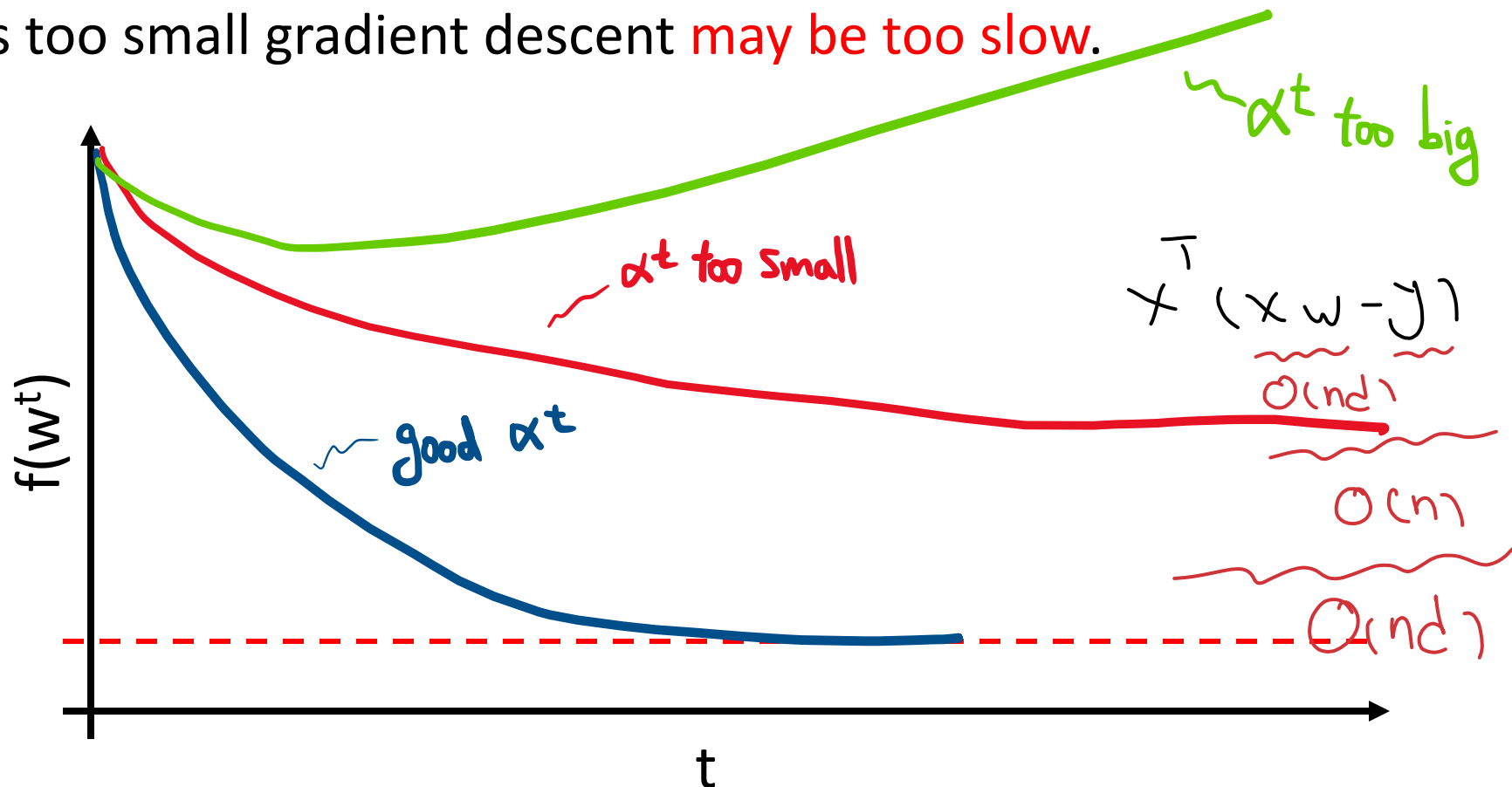
$\downarrow \times \nabla \leftarrow \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y})$
Too large
 $O(nd)$?

Too small

$O(nd)$ $O(nd)$ $O(n)$

Step Size Considerations

- The step size α^t must be set carefully for gradient descent to work.
 - If α^t is too large gradient descent may not converge.
 - If α^t is too small gradient descent may be too slow.



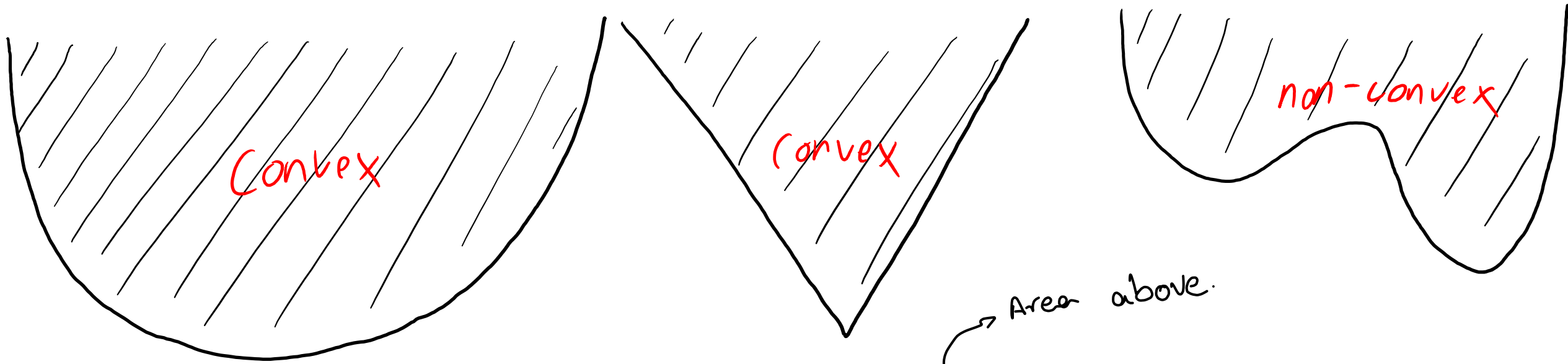
Beyond Gradient Descent

- There are many **variations on gradient descent**.
 - Methods employing a “line search” to choose the step-size.
 - “Conjugate” gradient and “accelerated” gradient methods.
 - Newton’s method (which uses second derivatives).
 - Quasi-Newton and Hessian-free Newton methods.
 - **Stochastic gradient descent** (later in course).
- This **course focuses on gradient descent and stochastic gradient**:
 - They’re simple and give reasonable solutions to most ML problems.
 - But the above can be faster for some applications.

Next Topic: Convex Functions

Convex Functions

- Is finding a 'w' with $\nabla f(w) = 0$ good enough?
 - Yes, for **convex functions**.

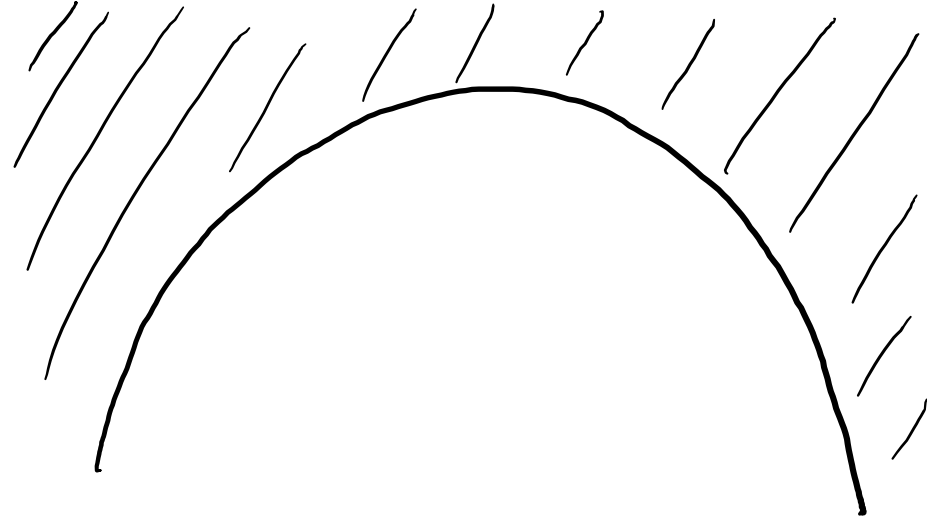


- A function is **convex** if the **area above the function is a convex set**.
 - All values between any two points above function stay above function.

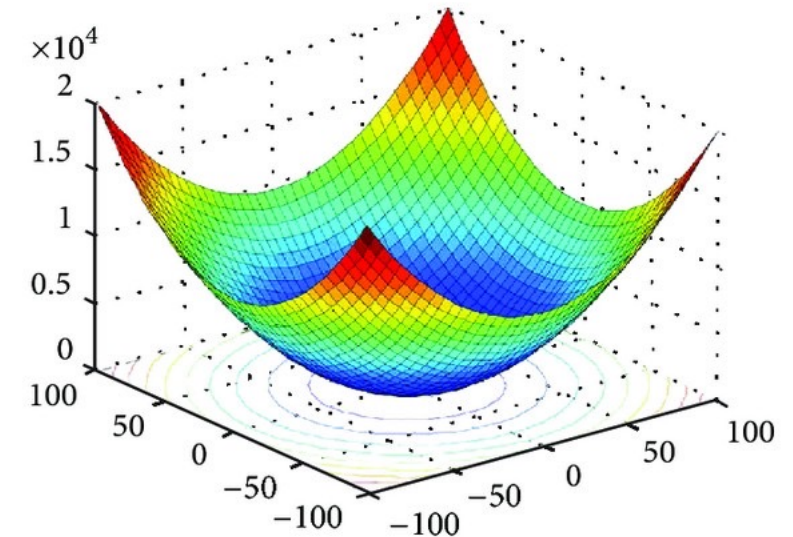
Convex Functions

- Example of **non-convex function**:

- In this case, we have $\nabla f(w)=0$ at a maximum.



- Example of **two-dimensional convex function**:



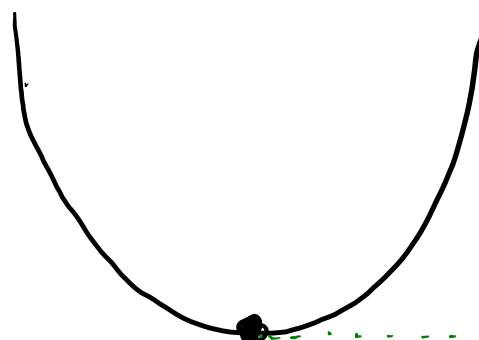
Convex Functions

- All 'w' with $\nabla f(w) = 0$ for convex functions are global minima.

Proof by contradiction:

Consider a local minimum

If this is not global minimum,
there must a smaller value.



By convexity we can move along line to global minimum and decrease objective.

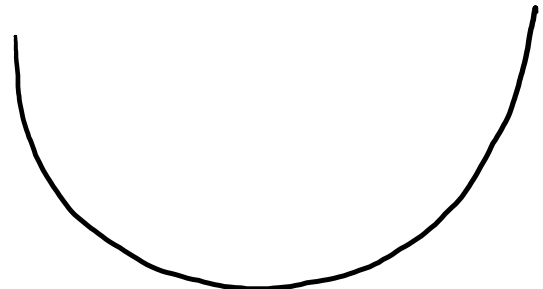
But this
contradicts that
we are at a
local minimum.

– Normal equations find a global minimum because least squares is convex.

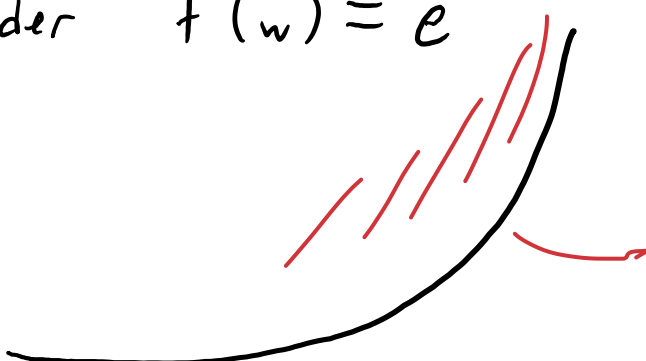
How do we know if a function is convex?

- Some useful tricks for showing a function is convex:
 - 1-variable, **twice-differentiable function is convex iff $f''(w) \geq 0$** for all ' w '.

Consider $f(w) = \frac{1}{2}aw^2$ for $a > 0$. We have $f'(w) = aw$
 and $f''(w) = a > 0$ *by assumption*



Consider $f(w) = e^w$. We have $f'(w) = e^w$
 and $f''(w) = e^w > 0$ *By definition of exponential function.*



doesn't have global min

How do we know if a function is convex?

OR line → Convex but no

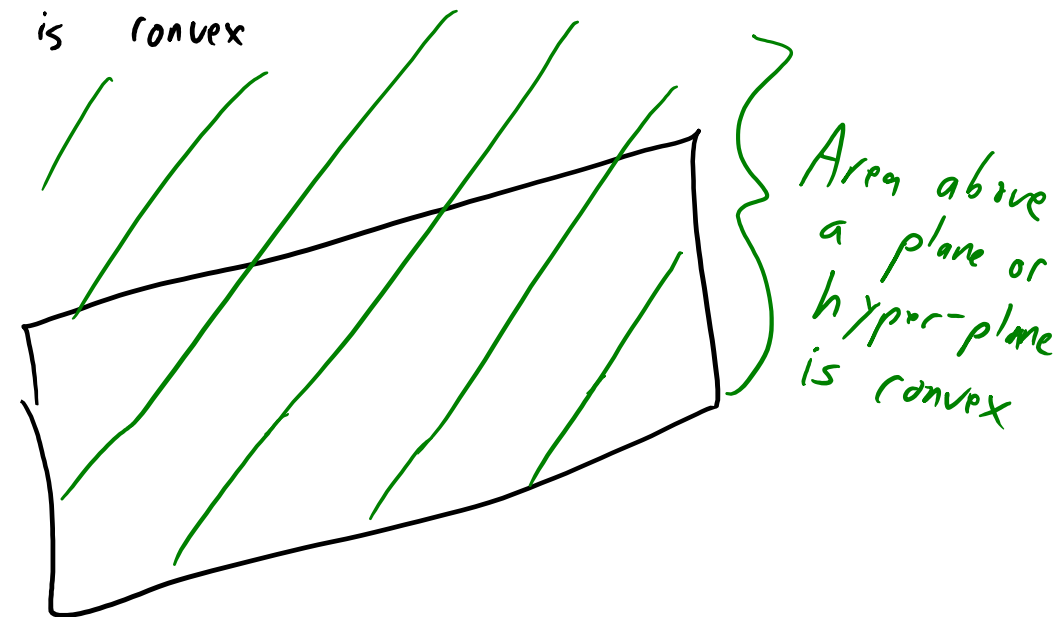
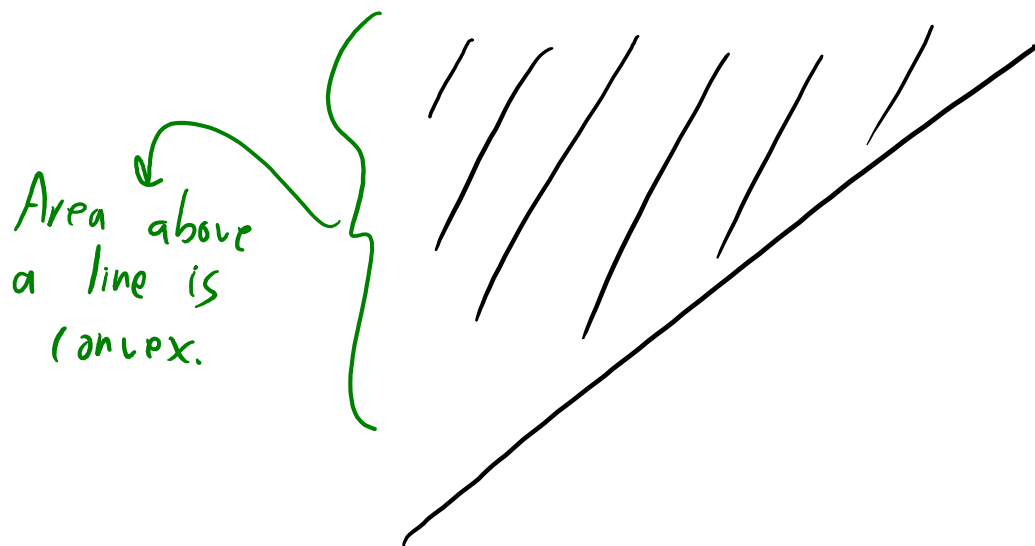
- Some useful tricks for showing a function is convex:
 - 1-variable, **twice-differentiable function is convex iff $f''(w) \geq 0$** for all 'w'. *Global min*
 - A convex function **multiplied by non-negative constant** is convex.

We showed that $f(w) = e^w$ is convex, so $f(w) = 10e^w$ is convex.

How do we know if a function is convex?

- Some useful tricks for showing a function is convex:
 - 1-variable, **twice-differentiable function is convex iff $f''(w) \geq 0$** for all 'w'.
 - A convex function **multiplied by non-negative constant** is convex.
 - **Linear functions** are convex.

$f(w) = w^T v$ for any vector 'v' is convex



How do we know if a function is convex?

- Some useful tricks for showing a function is convex:
 - 1-variable, twice-differentiable function is convex iff $f''(w) \geq 0$ for all 'w'.
 - A convex function multiplied by non-negative constant is convex.
 - Linear functions are convex.
 - Norms and squared norms are convex.

$\|w\|$, $\|w\|^2$, $\|w\|_1$, $\|w\|_\infty$, $\|w\|_1^2$, and so on are all convex.

How do we know if a function is convex?

- Some useful tricks for showing a function is convex:
 - 1-variable, twice-differentiable function is convex iff $f''(w) \geq 0$ for all 'w'.
 - A convex function multiplied by non-negative constant is convex.
 - Linear functions are convex.
 - Norms and squared norms are convex.
 - The sum of convex functions is a convex function.

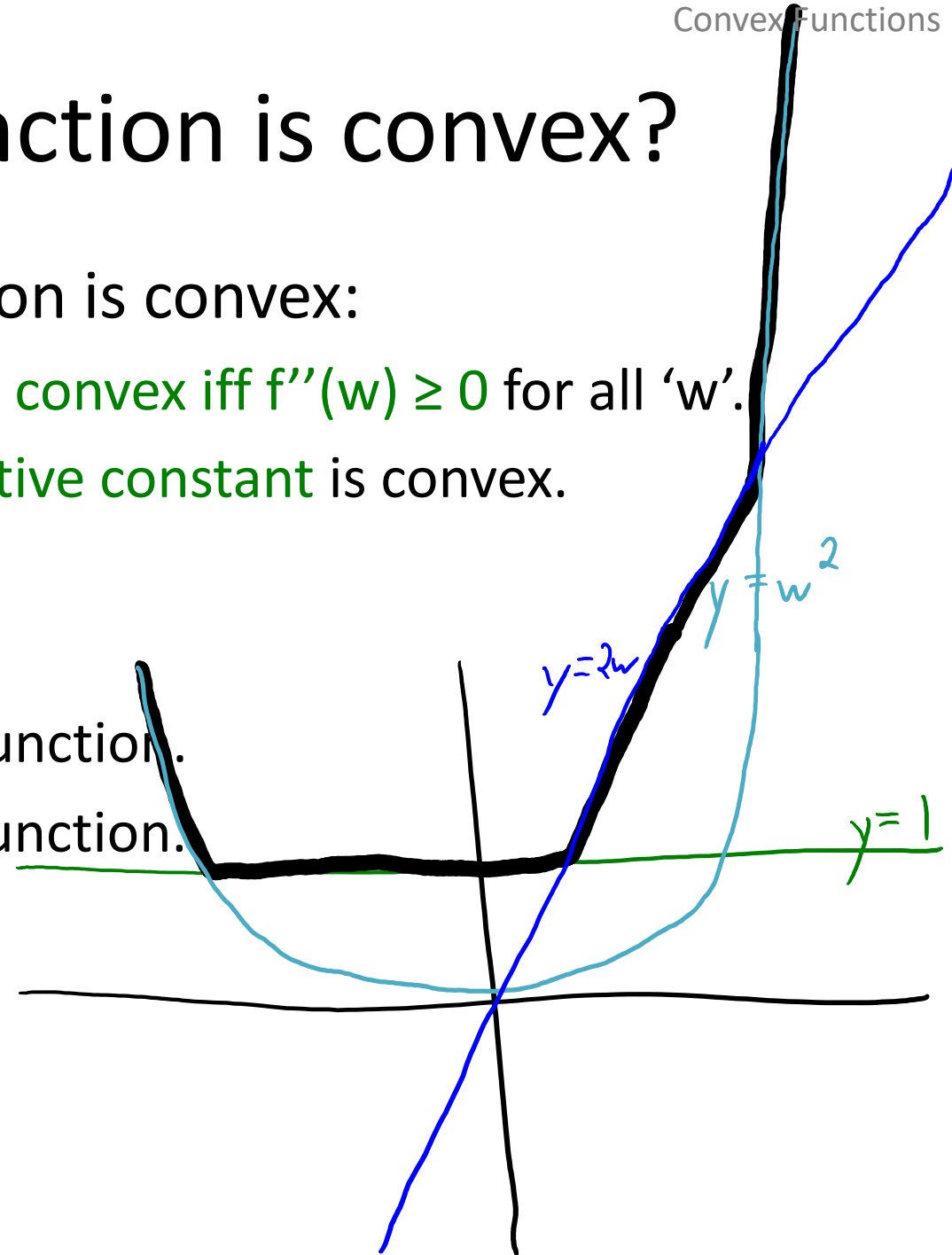
$$f(w) = \underbrace{10e^w}_{\text{From earlier}} + \underbrace{\frac{1}{2}}_{\text{positive constant}} \underbrace{\|w\|^2}_{\text{norm squared}} \text{ is convex}$$

How do we know if a function is convex?

- Some useful tricks for showing a function is convex:
 - 1-variable, **twice-differentiable function is convex iff $f''(w) \geq 0$** for all 'w'.
 - A convex function **multiplied by non-negative constant** is convex.
 - **Linear functions** are convex.
 - **Norms** and **squared norms** are convex.
 - The **sum of convex functions** is a convex function.
 - The **max of convex functions** is a convex function.

$$f(w) = \max \{ 1, 2w, w^2 \} \text{ is convex.}$$

$\uparrow \quad \uparrow \quad \uparrow$
 Convex



How do we know if a function is convex?

- Some useful tricks for showing a function is convex:
 - 1-variable, **twice-differentiable function is convex iff $f''(w) \geq 0$** for all 'w'.
 - A convex function **multiplied by non-negative constant** is convex.
 - **Linear functions** are convex.
 - **Norms and squared norms** are convex.
 - The **sum of convex functions** is a convex function.
 - The **max of convex functions** is a convex function.
 - **Composition of a convex function and a linear function** is convex.

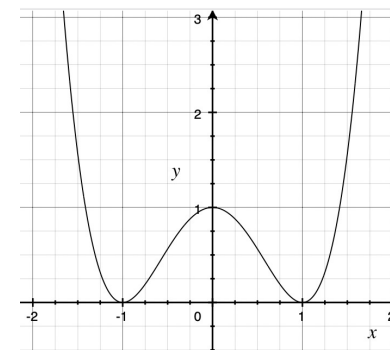
If $f(w) = g(\underbrace{Xw - y}_{\text{linear function (from } \mathbb{R}^d \text{ to } \mathbb{R}^n)})$ then 'f' is convex if 'g' is convex.

How do we know if a function is convex?

- Some useful tricks for showing a function is convex:
 - 1-variable, twice-differentiable function is convex iff $f''(w) \geq 0$ for all 'w'.
 - A convex function multiplied by non-negative constant is convex.
 - Linear functions are convex.
 - Norms and squared norms are convex.
 - The sum of convex functions is a convex function.
 - The max of convex functions is a convex function.
 - Composition of a convex function and a linear function is convex.
- But: not true that multiplication of convex functions is convex:
 - If $f(x)=x$ (convex) and $g(x)=x^2$ (convex), $f(x)g(x) = x^3$ (not convex).

How do we know if a function is convex?

- Some useful tricks for showing a function is convex:
 - 1-variable, twice-differentiable function is convex iff $f''(w) \geq 0$ for all 'w'.
 - A convex function multiplied by non-negative constant is convex.
 - Linear functions are convex.
 - Norms and squared norms are convex.
 - The sum of convex functions is a convex function.
 - The max of convex functions is a convex function.
 - Composition of a convex function and a linear function is convex.



$$(w^2 - 1)^2$$

- Also **not true that composition of convex with convex** is convex:

Even if 'f' is convex and 'g' is convex, $f(g(w))$ might not be convex.

E.g., w^2 is convex and $(w-1)^2$ is convex, but $(w^2-1)^2$ is not convex.

Example: Convexity of Linear Regression (Easy Way)

- Consider linear regression objective with squared error:

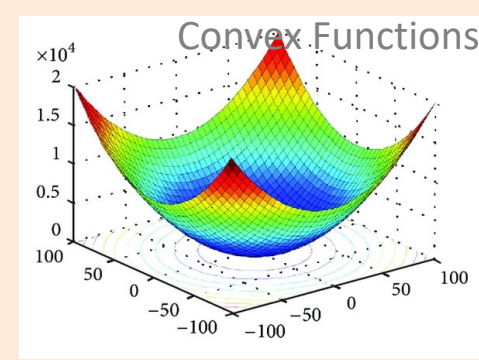
$$f(w) = \|Xw - y\|^2$$

- We can use that this is a **convex function composed with linear**:

$\left\{ \begin{array}{l} \text{Let } h(w) = Xw - y, \text{ which is a } \underline{\text{linear}} \text{ function ('d' inputs, 'n' outputs)} \\ \text{Let } g(r) = \|r\|^2, \text{ which is } \underline{\text{convex}} \text{ because it's a squared norm.} \\ \text{Then } f(w) = g(h(w)), \end{array} \right.$

which is **convex** because it's a **convex** function composed with a **linear** function

Convexity in Higher Dimensions



- Twice-differentiable 'd'-variable function is convex iff:
 - Eigenvalues of Hessian $\nabla^2 f(w)$ are non-negative for all 'w'.
- True for least squares where $\nabla^2 f(w) = X^T X$ for all 'w'.
 - See bonus slides for why $X^T X$ has non-negative eigenvalues.
- Unfortunately, sometimes it is hard to show convexity this way.
 - Usually easier to just use some of the rules as we did on the last slide.

Summary

- **Gradient descent** finds critical point of differentiable function.
 - Can be faster than normal equations for large 'd' values.
 - Finds global optimum if function is convex.
- **Convex functions:**
 - Set of functions with property that $\nabla f(w) = 0$ implies 'w' is a global min.
 - Can (usually) be identified using a few simple rules.
- **Next time:**
 - Linear regression without the outlier sensitivity...

Constraints, Continuity, Smoothness

- Sometimes we need to optimize with **constraints**:
 - Later we'll see “non-negative least squares”.

$$\min_{w \geq 0} \frac{1}{2} \sum_{i=1}^n (w^T x_i - y_i)^2$$

- A vector ‘w’ satisfying $w \geq 0$ (element-wise) is said to be “**feasible**”.
- Two factors affecting difficulty are **continuity** and **smoothness**.
 - **Continuous functions tend to be easier** than discontinuous functions.
 - **Smooth/differentiable functions tend to be easier** than non-smooth.
 - See the calculus review [here](#) if you haven't heard these words in a while.

Why use the negative gradient direction?

- For a twice-differentiable 'f', multivariable **Taylor expansion** gives:

$$f(w^{t+1}) = f(w^t) + \nabla f(w^t)^T (w^{t+1} - w^t) + \frac{1}{2} (w^{t+1} - w^t)^T \nabla^2 f(v) (w^{t+1} - w^t)$$

for some 'v' between w^{t+1} and w^t .

- If gradient can't change arbitrarily quickly, Hessian is bounded and:

$$f(w^{t+1}) = f(w^t) + \nabla f(w^t)^T (w^{t+1} - w^t) + O(\|w^{t+1} - w^t\|^2)$$

becomes negligible as w^{t+1} gets close to w^t

- But which **choice of w^{t+1} decreases 'f' the most?**

- As $\|w^{t+1} - w^t\|$ gets close to zero, the value of w^{t+1} minimizing $f(w^{t+1})$ in this formula converges to $(w^{t+1} - w^t) = -\alpha^t \nabla f(w^t)$ for some scalar α^t .

- So if we're moving a small amount, the optimal w^{t+1} is: $w^{t+1} = w^t - \alpha_t \nabla f(w^t)$ for some scalar α_t .

Normalized Steps

Question from class: "can we use $w^{t+1} = w^t - \frac{1}{\|\nabla f(w^t)\|} \nabla f(w^t)$ "

This will work for a while, but notice that

$$\begin{aligned} \|w^{t+1} - w^t\| &= \left\| \frac{1}{\|\nabla f(w^t)\|} \nabla f(w^t) \right\| \\ &= \frac{1}{\|\nabla f(w^t)\|} \|\nabla f(w^t)\| \\ &= 1 \end{aligned}$$

So the algorithm never converges

Optimizer “findMin” Details

? question ☆

stop following

99 views

The minimizer function

Hi all,

I'm just curious how the minimizers given to us works. Are there any resources can give us more details about it?

i the instructors' answer, *where instructors collectively construct a single answer*

It's just a basic gradient descent implementation with some clever guesses for the step-size.

The step-size on each iteration is initialized using the method from this classic paper (which works surprisingly well but we don't really know why except in two dimensions):

<http://pages.cs.wisc.edu/~swright/726/handouts/barzilai-borwein.pdf>

That step-size is evaluated using a standard condition ("Armijo condition") and then it fits a polynomial regression model based on the function and directional derivative values and tries the step-size minimizing this polynomial. Both these tricks are described in Nocedal and Wright's "Numerical Optimization" book.

edit

· good answer | 3

Updated 7 months ago by Mark Schmidt

? question @76   

stop following

81 v

3.3.8

Probably a stupid question, but when we do the gradient descent, is it required to calculate the loss value at each step? Are we allowed to save some middle results and reused them if we calculate both gradient and the loss? Thank you!

hw1

~ An instructor (Mark Schmidt) endorsed this question ~

Edit

undo good question | 1

Updated 8 months ago by  (Anon. Scale to class)



the instructors' answer, where instructors collectively construct a single answer

This is an important practical detail when you implement gradient descent, although there are few ways to interpret the question.

1. Technically, gradient descent never needs to compute the loss value. But you often compute it because it can help in checking whether you have a good step size.
2. Usually, some of the calculations involved in computing the gradient are also need to compute the function value. For example, a function of the form $f(Xw)$ has a gradient of the form $X^T f'(Xw)$, so if you evaluate the function and its gradient at the same 'w' you only want to compute Xw once. The function *findMin* asks for the function and gradient together so you can do things like this.
3. When you do a line-search, you often compute $f(\tilde{w})$ and possibly $\nabla f(\tilde{w})$ for some values $\tilde{w}$ that may later become w^{k+1} . You can use these quantities in the next iteration, instead of re-computing them. In *findMin*, it assumes that the line-search from the previous iteration gives you the current function and gradient value, so these do not need to be computed.

4. For many problems, there are tricks available where you only compute the loss/gradient with respect to a subset of the examples on each iteration, but then you still do the full update:

<https://arxiv.org/pdf/1309.2388v2.pdf>

Convexity, min, and argmin

- If a function is convex, then all critical points are global optima.
- However, **convex functions don't necessarily have critical points:**
 - For example, $f(x) = a^*x$, $f(x) = \exp(x)$, etc.
- Also, **more than one 'x' can achieve the global optimum:**
 - For example, $f(x) = c$ is minimized by any 'x'.

Example: Convexity of Linear Regression (Hard Way)

- Consider linear regression objective with squared error:

$$f(w) = \|Xw - y\|^2$$

- Twice-differentiable 'f' is convex if $\nabla^2 f(x)$ has eigenvalues ≥ 0 .
 - This is equivalent to saying $v^T \nabla^2 f(x) v \geq 0$ for all vectors v .
- The Hessian for least squares is $\nabla^2 f(x) = X^T X$.
 - See notes on Gradients and Hessians of quadratics on webpage.

- We have: $v^T \nabla^2 f(w) v = v^T X^T X v = (Xv)^T (Xv) = \|Xv\|^2 \geq 0$ (because norms are ≥ 0)

So it's convex